HARMONY

ROOMMATE CONFLICT RESOLUTION AND SHARED LIVING MANAGEMENT SYSTEM

COMPLETE SOURCE CODE LISTING & IMPLEMENTATION SPECIFICATIONS

Submitted in partial fulfillment of the requirements for the award of degree of
BACHELOR OF COMPUTER APPLICATIONS (BCA)
BCA 5th Semester | Course Code: BCA-508 (Minor Project)

Submitted By:
Lakshya Shukla
University Roll No: 240919106121 (BCA 5th Sem, Sec: B)
IMS Noida, Affiliated to CCS University, Meerut

Project Guide: Mr. Nihal Kumar, Assistant Professor
Review Panel: Panel 2 (Room DG4)
Date of Submission: 24th October 2026

INDEX OF IMPLEMENTATION MODULES

S.No	Module / File Name	Architectural Role
1	**Database Relational Schema (3NF DDL)** database/schema.sql	PostgreSQL tables, foreign key constraints, indexes
2	**Backend Server Listener** backend/src/server.js	Express HTTP server bootstrap and port listener
3	**Express Application & Middleware** backend/src/app.js	CORS, JSON body parser, route configuration, error handler
4	**Household API Controller** backend/src/routes/households.js	Household generation and code resolution endpoints
5	**Chore Engine API Controller** backend/src/routes/tasks.js	Task creation, effort point validation, toggle status
6	**Expense Splitting API Controller** backend/src/routes/expenses.js	Shared expense recording and participant split logic
7	**Frontend State Context & Failover Engine** frontend/src/context/AppContext.jsx	Client state management, net balance engine, offline fallback
8	**Roommate Live Dashboard Component** frontend/src/pages/DashboardPage.jsx	Interactive analytics, financial ledger, task boards
9	**Expense Management Component** frontend/src/pages/ExpensesPage.jsx	Shared bill entry modal, split previews, transaction history
10	**Chore Task Allocation Component** frontend/src/pages/TasksPage.jsx	Task list, effort-weighted assignment, status badges
11	**Household Admin Control Component** frontend/src/pages/AdminPage.jsx	Role-based controls, member eviction, ledger reset
12	**Vite Build Tooling Configuration** frontend/vite.config.js	Vite plugins, asset bundling, API reverse proxy
13	**Vercel Edge SPA Routing Rules** frontend/vercel.json	Production SPA URL rewrite configuration

1. Database Relational Schema (3NF DDL) *[database/schema.sql]*

PostgreSQL tables, foreign key constraints, indexes

```sql
-- ================================================================================
-- HARMONY: ROOMMATE CONFLICT RESOLUTION AND SHARED LIVING MANAGEMENT SYSTEM
-- BCA-508 Minor Project Relational Database Schema (PostgreSQL / Supabase)
-- ================================================================================

-- Enable UUID extension if not already enabled
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- --------------------------------------------------------------------------------
-- 1. PROFILES TABLE (Linked with Supabase Auth or Standalone)
-- --------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS profiles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    full_name VARCHAR(150) NOT NULL,
    avatar_url TEXT DEFAULT '',
    role VARCHAR(50) DEFAULT 'member', -- 'admin' or 'member'
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- --------------------------------------------------------------------------------
-- 2. HOUSEHOLDS TABLE (Shared Living Spaces)
-- --------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS households (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(150) NOT NULL,
    invite_code VARCHAR(10) UNIQUE NOT NULL,
    created_by UUID REFERENCES profiles(id) ON DELETE SET NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- --------------------------------------------------------------------------------
-- 3. HOUSEHOLD_MEMBERS TABLE (Bridge / Junction Table resolving M:N)
-- --------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS household_members (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    role VARCHAR(50) DEFAULT 'member', -- 'admin' or 'member'
    joined_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT unique_household_user UNIQUE (household_id, user_id)
);

-- --------------------------------------------------------------------------------
-- 4. TASKS TABLE (Chore Management & Gamified Points)
-- --------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
    title VARCHAR(200) NOT NULL,
    description TEXT DEFAULT '',
    category VARCHAR(100) DEFAULT 'General', -- 'Kitchen', 'Cleaning', 'Groceries', 'Maintenance'
    points INTEGER DEFAULT 10 CHECK (points >= 0),
    assigned_to UUID REFERENCES profiles(id) ON DELETE SET NULL,
    completed_by UUID REFERENCES profiles(id) ON DELETE SET NULL,
    status VARCHAR(50) DEFAULT 'PENDING', -- 'PENDING', 'COMPLETED'
    due_date DATE,
    completed_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- --------------------------------------------------------------------------------
-- 5. EXPENSES TABLE (Master Shared Expenditure Records)
```

```sql
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS expenses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
    paid_by UUID REFERENCES profiles(id) ON DELETE SET NULL,
    description VARCHAR(255) NOT NULL,
    amount DECIMAL(10, 2) NOT NULL CHECK (amount > 0),
    category VARCHAR(100) DEFAULT 'General', -- 'Utilities', 'Food', 'Internet', 'Supplies'
    expense_date DATE DEFAULT CURRENT_DATE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- ------------------------------------------------------------------------
-- 6. EXPENSE_SPLITS TABLE (Junction Table: Roommates vs Expenses)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS expense_splits (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    expense_id UUID NOT NULL REFERENCES expenses(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    share_amount DECIMAL(10, 2) NOT NULL CHECK (share_amount >= 0),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT unique_expense_user UNIQUE (expense_id, user_id)
);

-- ------------------------------------------------------------------------
-- DATABASE INDEXES (Performance & Query Optimization)
-- ------------------------------------------------------------------------
CREATE INDEX IF NOT EXISTS idx_households_invite_code ON households(invite_code);
CREATE INDEX IF NOT EXISTS idx_members_household ON household_members(household_id);
CREATE INDEX IF NOT EXISTS idx_members_user ON household_members(user_id);
CREATE INDEX IF NOT EXISTS idx_tasks_household ON tasks(household_id);
CREATE INDEX IF NOT EXISTS idx_tasks_status ON tasks(status);
CREATE INDEX IF NOT EXISTS idx_expenses_household ON expenses(household_id);
CREATE INDEX IF NOT EXISTS idx_splits_expense ON expense_splits(expense_id);
CREATE INDEX IF NOT EXISTS idx_splits_user ON expense_splits(user_id);
```

2. Backend Server Listener *[backend/src/server.js]*

Express HTTP server bootstrap and port listener

```javascript
const app = require('./app');
require('dotenv').config();

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`=====================================================`);
  console.log(`HARMONY Backend Server running on port ${PORT}`);
  console.log(`Health Check: http://localhost:${PORT}/api/health`);
  console.log(`Dashboard API: http://localhost:${PORT}/api/dashboard/h1`);
  console.log(`=====================================================`);
});
```

3. Express Application & Middleware *[backend/src/app.js]*

CORS, JSON body parser, route configuration, error handler

```javascript
const express = require('express');
const cors = require('cors');
const morgan = require('morgan');

const healthRoutes = require('./routes/health');
const householdRoutes = require('./routes/households');
const taskRoutes = require('./routes/tasks');
const expenseRoutes = require('./routes/expenses');
```

```
const dashboardRoutes = require('./routes/dashboard');
const adminRoutes = require('./routes/admin');

const app = express();

// Middlewares
app.use(cors({
  origin: '*', // Allows Vercel frontend and localhost
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));

app.use(express.json());
app.use(morgan('dev'));

// API Routes
app.use('/api/health', healthRoutes);
app.use('/api/households', householdRoutes);
app.use('/api/tasks', taskRoutes);
app.use('/api/expenses', expenseRoutes);
app.use('/api/dashboard', dashboardRoutes);
app.use('/api/admin', adminRoutes);

// Root route
app.get('/', (req, res) => {
  res.json({
    message: 'HARMONY REST API Server is active.',
    health: '/api/health',
    docs: 'BCA-508 Minor Project'
  });
});

// 404 Handler
app.use((req, res) => {
  res.status(404).json({ error: 'Endpoint not found', path: req.originalUrl });
});

// Global Error Handler
app.use((err, req, res, next) => {
  console.error('[Error]', err);
  res.status(500).json({ error: 'Internal Server Error', message: err.message });
});

module.exports = app;
```

4. Household API Controller *[backend/src/routes/households.js]*

Household generation and code resolution endpoints

```
const express = require('express');
const router = express.Router();
const mockStore = require('../data/mockStore');

// GET household details
router.get('/:id', (req, res) => {
  const h = mockStore.getHousehold(req.params.id);
  if (!h) return res.status(404).json({ error: 'Household not found' });
  const members = mockStore.getMembers(h.id);
  res.json({ household: h, members });
});

// POST create household
router.post('/', (req, res) => {
  const { name, userId } = req.body;
  if (!name) return res.status(400).json({ error: 'Household name is required' });
  const created = mockStore.createHousehold(name, userId || 'u1');
```

```
  res.status(201).json(created);
});

// POST join household via invite code
router.post('/join', (req, res) => {
  const { inviteCode, userId } = req.body;
  if (!inviteCode) return res.status(400).json({ error: 'Invite code is required' });
  const result = mockStore.joinHhousehold(inviteCode, userId || 'u1');
  if (result.error) return res.status(400).json(result);
  res.json(result);
});

// GET household members
router.get('/:id/members', (req, res) => {
  const members = mockStore.getMembers(req.params.id);
  res.json(members);
});

module.exports = router;
```

5. Chore Engine API Controller *[backend/src/routes/tasks.js]*

Task creation, effort point validation, toggle status

```
const express = require('express');
const router = express.Router();
const mockStore = require('../data/mockStore');

// GET all tasks for a household
router.get('/', (req, res) => {
  const householdId = req.query.householdId || 'h1';
  const tasks = mockStore.getTasks(householdId);
  res.json(tasks);
});

// POST create chore task
router.post('/', (req, res) => {
  const { title, description, category, points, assigned_to, due_date, household_id } = req.body;
  if (!title) return res.status(400).json({ error: 'Title is required' });
  const task = mockStore.createTask({
    title,
    description,
    category,
    points,
    assigned_to,
    due_date,
    household_id: household_id || 'h1'
  });
  res.status(201).json(task);
});

// PATCH toggle complete status
router.patch('/:id/toggle', (req, res) => {
  const { userId } = req.body;
  const updated = mockStore.toggleTaskComplete(req.params.id, userId || 'u1');
  if (!updated) return res.status(404).json({ error: 'Task not found' });
  res.json(updated);
});

// DELETE task
router.delete('/:id', (req, res) => {
  const success = mockStore.deleteTask(req.params.id);
  if (!success) return res.status(404).json({ error: 'Task not found' });
  res.json({ message: 'Task deleted successfully' });
});

```

```
module.exports = router;
```

6. Expense Splitting API Controller *[backend/src/routes/expenses.js]*

Shared expense recording and participant split logic

```
const express = require('express');
const router = express.Router();
const mockStore = require('../data/mockStore');

// GET all expenses for a household
router.get('/', (req, res) => {
  const householdId = req.query.householdId || 'h1';
  const expenses = mockStore.getExpenses(householdId);
  res.json(expenses);
});

// POST create shared expense with equal split
router.post('/', (req, res) => {
  const { description, amount, category, paid_by, participants, expense_date, household_id } = req.body;
  if (!description || !amount) {
    return res.status(400).json({ error: 'Description and amount are required' });
  }
  const expense = mockStore.createExpense({
    description,
    amount,
    category,
    paid_by: paid_by || 'u1',
    participants: participants || ['u1', 'u2', 'u3', 'u4'],
    expense_date,
    household_id: household_id || 'h1'
  });
  res.status(201).json(expense);
});

// DELETE expense
router.delete('/:id', (req, res) => {
  const success = mockStore.deleteExpense(req.params.id);
  if (!success) return res.status(404).json({ error: 'Expense not found' });
  res.json({ message: 'Expense deleted successfully' });
});

module.exports = router;
```

7. Frontend State Context & Failover Engine *[frontend/src/context/AppContext.jsx]*

Client state management, net balance engine, offline fallback

```
import React, { createContext, useContext, useState, useEffect } from 'react';

const AppContext = createContext();

const API_BASE = import.meta.env.VITE_API_URL ? `${import.meta.env.VITE_API_URL.replace(/\/$/, '')}/api` : '/api';

// Initial clean demo flatmates for instant 0ms rendering and zero-friction evaluation
const INITIAL_USERS = [
  { id: 'u1', name: 'Lakshya Shukla', email: 'lakshya@harmony.app', role: 'admin', avatarColor: 'from-indigo-500 to-purple-600', initials: 'LS' },
  { id: 'u2', name: 'Rahul Verma', email: 'rahul@harmony.app', role: 'member', avatarColor: 'from-emerald-500 to-teal-600', initials: 'RV' },
  { id: 'u3', name: 'Aman Gupta', email: 'aman@harmony.app', role: 'member', avatarColor: 'from-amber-500 to-orange-600', initials: 'AG' },
  { id: 'u4', name: 'Rohit Kumar', email: 'rohit@harmony.app', role: 'member', avatarColor: 'from-rose-500 to-pink-600', initials: 'RK' }
```

```
];

const INITIAL_HOUSEHOLD = {
  id: 'h1',
  name: 'Green Residency (Flat 402)',
  invite_code: 'GRN101'
};

const INITIAL_TASKS = [
  { id: 't1', household_id: 'h1', title: 'Deep Clean Kitchen & Counters', category: 'Kitchen', points: 25, assigned_to: 'u1', assigned_to_name: 'Lakshya', completed_by: 'u1', status: 'COMPLETED', due_date: '2026-10-05' },
  { id: 't2', household_id: 'h1', title: 'Living Room Vacuum & Mopping', category: 'Cleaning', points: 30, assigned_to: 'u2', assigned_to_name: 'Rahul', completed_by: 'u2', status: 'COMPLETED', due_date: '2026-10-06' },
  { id: 't3', household_id: 'h1', title: 'Weekly Blinkit Superstore Grocery Run', category: 'Groceries', points: 20, assigned_to: 'u3', assigned_to_name: 'Aman', completed_by: 'u3', status: 'COMPLETED', due_date: '2026-10-06' },
  { id: 't4', household_id: 'h1', title: 'Dispose Waste & Replace Bin Liners', category: 'Cleaning', points: 15, assigned_to: 'u4', assigned_to_name: 'Rohit', completed_by: 'u4', status: 'COMPLETED', due_date: '2026-10-07' },
  { id: 't5', household_id: 'h1', title: 'Common Bathroom Sanitisation', category: 'Cleaning', points: 35, assigned_to: 'u1', assigned_to_name: 'Lakshya', completed_by: 'u1', status: 'COMPLETED', due_date: '2026-10-07' },
  { id: 't6', household_id: 'h1', title: 'Refill 20L Drinking Water Dispensers', category: 'Maintenance', points: 20, assigned_to: 'u2', assigned_to_name: 'Rahul', completed_by: 'u2', status: 'COMPLETED', due_date: '2026-10-08' },
  { id: 't7', household_id: 'h1', title: 'Wash Cooking Vessels & Dinner Utensils', category: 'Kitchen', points: 20, assigned_to: 'u3', assigned_to_name: 'Aman', completed_by: 'u3', status: 'COMPLETED', due_date: '2026-10-08' },
  { id: 't8', household_id: 'h1', title: 'Clean Refrigerator & Expired Food', category: 'Kitchen', points: 25, assigned_to: 'u4', assigned_to_name: 'Rohit', completed_by: 'u4', status: 'COMPLETED', due_date: '2026-10-08' },
  { id: 't9', household_id: 'h1', title: 'Restock Toiletries, Detergent & Handwash', category: 'Groceries', points: 15, assigned_to: 'u1', assigned_to_name: 'Lakshya', completed_by: null, status: 'PENDING', due_date: '2026-10-10' },
  { id: 't10', household_id: 'h1', title: 'Balcony Sweep & Water Living Plants', category: 'Cleaning', points: 15, assigned_to: 'u2', assigned_to_name: 'Rahul', completed_by: null, status: 'PENDING', due_date: '2026-10-11' },
  { id: 't11', household_id: 'h1', title: 'Clean AC Filters & Master Bedroom Vent', category: 'Maintenance', points: 30, assigned_to: 'u3', assigned_to_name: 'Aman', completed_by: null, status: 'PENDING', due_date: '2026-10-12' },
  { id: 't12', household_id: 'h1', title: 'Flatten Delivery Cardboard & Newspaper', category: 'Maintenance', points: 20, assigned_to: 'u4', assigned_to_name: 'Rohit', completed_by: null, status: 'PENDING', due_date: '2026-10-13' }
];

const INITIAL_EXPENSES = [
  { id: 'e1', household_id: 'h1', paid_by: 'u1', paid_by_name: 'Lakshya Shukla', description: 'September Electricity & Power Bill', amount: 2400.0, category: 'Utilities', expense_date: '2026-10-03', splits_count: 4 },
  { id: 'e2', household_id: 'h1', paid_by: 'u2', paid_by_name: 'Rahul Verma', description: 'Airtel Xstream Fiber (300 Mbps Unlimited)', amount: 1200.0, category: 'Internet', expense_date: '2026-10-04', splits_count: 4 },
  { id: 'e3', household_id: 'h1', paid_by: 'u3', paid_by_name: 'Aman Gupta', description: 'Blinkit Bulk Household Ration & Dairy', amount: 3600.0, category: 'Food', expense_date: '2026-10-05', splits_count: 4 },
  { id: 'e4', household_id: 'h1', paid_by: 'u4', paid_by_name: 'Rohit Kumar', description: 'HP Gas Cooking Cylinder Refill', amount: 1100.0, category: 'Utilities', expense_date: '2026-10-06', splits_count: 4 },
  { id: 'e5', household_id: 'h1', paid_by: 'u1', paid_by_name: 'Lakshya Shukla', description: 'Urban Company RO Service & Filter Replacement', amount: 1500.0, category: 'Supplies', expense_date: '2026-10-07', splits_count: 4 }
];

const INITIAL_DASHBOARD = {
  kpis: {
    pendingChores: 4,
    completedChores: 8,
    totalExpenses: 9800,
    totalMembers: 4
  },
  pointsData: [
    { id: 'u1', name: 'Lakshya', full_name: 'Lakshya Shukla', points: 60 },
    { id: 'u2', name: 'Rahul', full_name: 'Rahul Verma', points: 50 },
    { id: 'u3', name: 'Aman', full_name: 'Aman Gupta', points: 40 },
```

```
    { id: 'u4', name: 'Rohit', full_name: 'Rohit Kumar', points: 40 }
  ],
  balanceData: [
    { id: 'u1', name: 'Lakshya Shukla', totalPaid: 3900, totalOwed: 2450, netBalance: 1450 },
    { id: 'u2', name: 'Rahul Verma', totalPaid: 1200, totalOwed: 2450, netBalance: -1250 },
    { id: 'u3', name: 'Aman Gupta', totalPaid: 3600, totalOwed: 2450, netBalance: 1150 },
    { id: 'u4', name: 'Rohit Kumar', totalPaid: 1100, totalOwed: 2450, netBalance: -1350 }
  ],
  recentChores: INITIAL_TASKS.slice(0, 5),
  recentExpenses: INITIAL_EXPENSES.slice(0, 5)
};

export const AppProvider = ({ children }) => {
  const [demoUsers] = useState(INITIAL_USERS);
  const [currentUser, setCurrentUser] = useState(INITIAL_USERS[0]);
  const [household, setHousehold] = useState(INITIAL_HOUSEHOLD);
  const [tasks, setTasks] = useState(INITIAL_TASKS);
  const [expenses, setExpenses] = useState(INITIAL_EXPENSES);
  const [dashboardData, setDashboardData] = useState(INITIAL_DASHBOARD);
  const [loading, setLoading] = useState(false);
  const [isRefreshing, setIsRefreshing] = useState(false);
  const [apiConnected, setApiConnected] = useState(false);

  // Recalculate local dashboard metrics instantly
  const computeLocalDashboard = (currentTasks, currentExpenses) => {
    const pendingChores = currentTasks.filter(t => t.status === 'PENDING').length;
    const completedChores = currentTasks.filter(t => t.status === 'COMPLETED').length;
    const totalExpenses = currentExpenses.reduce((sum, e) => sum + Number(e.amount), 0);

    const pointsData = demoUsers.map(u => ({
      id: u.id,
      name: u.name.split(' ')[0],
      full_name: u.name,
      points: currentTasks
        .filter(t => t.status === 'COMPLETED' && (t.completed_by === u.id || t.assigned_to === u.id))
        .reduce((sum, t) => sum + (t.points || 0), 0)
    }));

    const splitPerUser = currentExpenses.length > 0 ? totalExpenses / demoUsers.length : 0;
    const balanceData = demoUsers.map(u => {
      const totalPaid = currentExpenses
        .filter(e => e.paid_by === u.id)
        .reduce((sum, e) => sum + Number(e.amount), 0);
      return {
        id: u.id,
        name: u.name,
        totalPaid,
        totalOwed: splitPerUser,
        netBalance: Math.round((totalPaid - splitPerUser) * 100) / 100
      };
    });

    return {
      kpis: { pendingChores, completedChores, totalExpenses, totalMembers: demoUsers.length },
      pointsData,
      balanceData,
      recentChores: currentTasks.slice(0, 5),
      recentExpenses: currentExpenses.slice(0, 5)
    };
  };

  // Fetch all state from live API with automatic fallback
  const refreshData = async () => {
    setIsRefreshing(true);
    try {
      const controller = new AbortController();
      const timeoutId = setTimeout(() => controller.abort(), 2500);
```

```
122
123       const [dashRes, tasksRes, expRes] = await Promise.all([
124         fetch(`${API_BASE}/dashboard/${household.id}`, { signal: controller.signal }),
125         fetch(`${API_BASE}/tasks?householdId=${household.id}`, { signal: controller.signal }),
126         fetch(`${API_BASE}/expenses?householdId=${household.id}`, { signal: controller.signal })
127       ]);
128       clearTimeout(timeoutId);
129
130       if (dashRes.ok && tasksRes.ok && expRes.ok) {
131         const dash = await dashRes.json();
132         const t = await tasksRes.json();
133         const e = await expRes.json();
134         setDashboardData(dash);
135         setTasks(t);
136         setExpenses(e);
137         setApiConnected(true);
138       }
139     } catch (err) {
140       setApiConnected(false);
141     } finally {
142       setTimeout(() => setIsRefreshing(false), 400);
143     }
144   };
145
146   useEffect(() => {
147     refreshData();
148   }, [household.id]);
149
150   // Create Task
151   const addTask = async (taskData) => {
152     try {
153       if (apiConnected) {
154         const res = await fetch(`${API_BASE}/tasks`, {
155           method: 'POST',
156           headers: { 'Content-Type': 'application/json' },
157           body: JSON.stringify({ ...taskData, household_id: household.id })
158         });
159         if (res.ok) {
160           await refreshData();
161           return true;
162         }
163       }
164     } catch (e) {
165       console.warn('API error, falling back to local state:', e);
166     }
167
168     const assignedUser = demoUsers.find(u => u.id === taskData.assigned_to) || currentUser;
169     const newTask = {
170       id: 't_' + Date.now(),
171       household_id: household.id,
172       title: taskData.title,
173       category: taskData.category || 'General',
174       points: Number(taskData.points) || 20,
175       assigned_to: assignedUser.id,
176       assigned_to_name: assignedUser.name.split(' ')[0],
177       completed_by: null,
178       status: 'PENDING',
179       due_date: taskData.due_date || new Date().toISOString().split('T')[0]
180     };
181     const updated = [newTask, ...tasks];
182     setTasks(updated);
183     setDashboardData(computeLocalDashboard(updated, expenses));
184     return true;
185   };
186
187   // Toggle Task Completion
188   const toggleTask = async (taskId) => {
```

```
    try {
      if (apiConnected) {
        const res = await fetch(`${API_BASE}/tasks/${taskId}/toggle`, {
          method: 'PATCH',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ userId: currentUser.id })
        });
        if (res.ok) {
          await refreshData();
          return true;
        }
      }
    } catch (e) {
      console.warn('API error, falling back to local toggle:', e);
    }

    const updated = tasks.map(t => {
      if (t.id === taskId) {
        const isDone = t.status === 'COMPLETED';
        return {
          ...t,
          status: isDone ? 'PENDING' : 'COMPLETED',
          completed_by: isDone ? null : currentUser.id
        };
      }
      return t;
    });
    setTasks(updated);
    setDashboardData(computeLocalDashboard(updated, expenses));
    return true;
  };

  // Delete Task
  const deleteTask = async (taskId) => {
    try {
      if (apiConnected) {
        const res = await fetch(`${API_BASE}/tasks/${taskId}`, { method: 'DELETE' });
        if (res.ok) {
          await refreshData();
          return true;
        }
      }
    } catch (e) {
      console.warn('API error, falling back to local delete:', e);
    }

    const updated = tasks.filter(t => t.id !== taskId);
    setTasks(updated);
    setDashboardData(computeLocalDashboard(updated, expenses));
    return true;
  };

  // Add Expense with equal split
  const addExpense = async (expenseData) => {
    try {
      if (apiConnected) {
        const res = await fetch(`${API_BASE}/expenses`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
            ...expenseData,
            paid_by: currentUser.id,
            household_id: household.id
          })
        });
        if (res.ok) {
          await refreshData();
```

```
          return true;
        }
      }
    } catch (e) {
      console.warn('API error, falling back to local expense:', e);
    }

    const newExp = {
      id: 'e_' + Date.now(),
      household_id: household.id,
      paid_by: currentUser.id,
      paid_by_name: currentUser.name,
      description: expenseData.description,
      amount: parseFloat(expenseData.amount),
      category: expenseData.category || 'General',
      expense_date: expenseData.expense_date || new Date().toISOString().split('T')[0],
      splits_count: demoUsers.length
    };
    const updated = [newExp, ...expenses];
    setExpenses(updated);
    setDashboardData(computeLocalDashboard(tasks, updated));
    return true;
  };

  // Delete Expense
  const deleteExpense = async (expenseId) => {
    try {
      if (apiConnected) {
        const res = await fetch(`${API_BASE}/expenses/${expenseId}`, { method: 'DELETE' });
        if (res.ok) {
          await refreshData();
          return true;
        }
      }
    } catch (e) {
      console.warn('API error, falling back to local delete:', e);
    }

    const updated = expenses.filter(e => e.id !== expenseId);
    setExpenses(updated);
    setDashboardData(computeLocalDashboard(tasks, updated));
    return true;
  };

  // Instant Debt Settle-Up (Clear or log settlement payment)
  const settleUp = (debtorName, amount) => {
    const settlementExpense = {
      id: 'e_settle_' + Date.now(),
      household_id: household.id,
      paid_by: currentUser.id,
      paid_by_name: currentUser.name,
      description: `Settled payment with ${debtorName}`,
      amount: Number(amount) || 500,
      category: 'Settlement',
      expense_date: new Date().toISOString().split('T')[0],
      splits_count: 2
    };
    const updated = [settlementExpense, ...expenses];
    setExpenses(updated);
    setDashboardData(computeLocalDashboard(tasks, updated));
    return true;
  };

  // Switch demo roommate (Lakshya / Rahul / Aman / Rohit)
  const switchUser = (userId) => {
    const found = demoUsers.find(u => u.id === userId);
    if (found) setCurrentUser(found);
```

```
  };

  return (
    <AppContext.Provider
      value={{
        currentUser,
        demoUsers,
        household,
        tasks,
        expenses,
        dashboardData,
        loading,
        isRefreshing,
        apiConnected,
        refreshData,
        addTask,
        toggleTask,
        deleteTask,
        addExpense,
        deleteExpense,
        settleUp,
        switchUser
      }}
    >
      {children}
    </AppContext.Provider>
  );
};

export const useApp = () => useContext(AppContext);
```

8. Roommate Live Dashboard Component *[frontend/src/pages/DashboardPage.jsx]*

Interactive analytics, financial ledger, task boards

```
import React, { useState } from 'react';
import { useApp } from '../context/AppContext';
import {
  Receipt,
  CheckCircle2,
  Plus,
  ArrowUpRight,
  ArrowDownLeft,
  Sparkles,
  Zap,
  TrendingUp,
  CreditCard,
  Check,
  Clock,
  Trash2,
  Users
} from 'lucide-react';
import {
  BarChart,
  Bar,
  XAxis,
  YAxis,
  Tooltip,
  ResponsiveContainer,
  Cell
} from 'recharts';
import AddChoreModal from '../components/AddChoreModal';
import AddExpenseModal from '../components/AddExpenseModal';

export default function DashboardPage() {
  const { dashboardData, currentUser, household, loading, toggleTask, settleUp } = useApp();
```

```
  const [choreModalOpen, setChoreModalOpen] = useState(false);
  const [expenseModalOpen, setExpenseModalOpen] = useState(false);
  const [activeTab, setActiveTab] = useState('expenses'); // 'expenses' or 'chores'
  const [settledMessage, setSettledMessage] = useState(null);

  if (loading || !dashboardData) {
    return (
      <div className="flex items-center justify-center min-h-[60vh]">
        <div className="flex flex-col items-center space-y-3 text-slate-400">
          <div className="w-10 h-10 border-4 border-indigo-500 border-t-transparent rounded-full animate-
spin"></div>
          <span className="text-sm font-medium">Loading household ledger...</span>
        </div>
      </div>
    );
  }

  const { kpis, pointsData, balanceData, recentChores, recentExpenses } = dashboardData;

  // Find current active user's financial balance
  const myBalance = balanceData.find(b => b.id === currentUser.id) || {
    totalPaid: 0,
    totalOwed: 0,
    netBalance: 0
  };

  const isOwed = myBalance.netBalance >= 0;
  const absBalance = Math.abs(myBalance.netBalance).toLocaleString('en-IN', {
    maximumFractionDigits: 2
  });

  const handleSettle = (userName, amount) => {
    settleUp(userName, amount);
    setSettledMessage(`Settlement recorded with ${userName}! Ledger balanced.`);
    setTimeout(() => setSettledMessage(null), 4000);
  };

  return (
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8 space-y-8">

      {/* Settlement Notification Toast */}
      {settledMessage && (
        <div className="p-4 rounded-2xl bg-emerald-500/10 border border-emerald-500/20 text-emerald-400 text-
sm font-semibold flex items-center justify-between animate-in fade-in">
          <div className="flex items-center space-x-2">
            <Check className="w-4 h-4" />
            <span>{settledMessage}</span>
          </div>
          <button onClick={() => setSettledMessage(null)} className="text-xs text-slate-400 hover:text-
white">✕</button>
        </div>
      )}

      {/* Top Welcome Banner & Fast Actions */}
      <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-4 bg-[#11182B]
p-6 sm:p-8 rounded-3xl border border-white/10 shadow-2xl relative overflow-hidden">
        <div className="absolute top-0 right-0 -mr-16 -mt-16 w-64 h-64 bg-indigo-500/10 rounded-full blur-3xl
pointer-events-none"></div>

        <div>
          <div className="flex items-center space-x-2.5">
            <h1 className="text-2xl sm:text-3xl font-extrabold text-white tracking-tight font-display">
              Hey, {currentUser.name.split(' ')[0]} 👋
            </h1>
            <span className="px-2.5 py-0.5 rounded-full text-[11px] font-bold bg-indigo-500/10 text-indigo-300
border border-indigo-500/20">
              {currentUser.role === 'admin' ? 'Flat Manager' : 'Flatmate'}
```

```
              </span>
            </div>
            <p className="mt-1 text-sm text-slate-400 flex items-center space-x-1.5">
              <span>Household: <strong className="text-slate-200">{household.name}</strong></span>
              <span>·</span>
              <span className="font-mono text-indigo-300 font-bold">Code: {household.invite_code}</span>
            </p>
          </div>

          {/* Quick Action Modals */}
          <div className="flex items-center space-x-3 w-full sm:w-auto">
            <button
              onClick={() => setExpenseModalOpen(true)}
              className="flex-1 sm:flex-none flex items-center justify-center space-x-2 px-5 py-3 rounded-2xl
bg-gradient-to-r from-emerald-500 to-teal-600 hover:from-emerald-600 hover:to-teal-700 text-white font-bold text-xs
sm:text-sm shadow-lg shadow-emerald-500/20 transition-all transform hover:-translate-y-0.5"
            >
              <Plus className="w-4 h-4" />
              <span>Add Bill</span>
            </button>

            <button
              onClick={() => setChoreModalOpen(true)}
              className="flex-1 sm:flex-none flex items-center justify-center space-x-2 px-5 py-3 rounded-2xl
bg-[#172038] hover:bg-[#1E2B4C] border border-white/10 text-slate-200 hover:text-white font-bold text-xs sm:text-sm
transition-all"
            >
              <CheckCircle2 className="w-4 h-4 text-indigo-400" />
              <span>Add Task</span>
            </button>
          </div>
        </div>

        {/* 4 Clean Metric Cards */}
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4 sm:gap-6">

          {/* Card 1: Your Clear Balance (Simplified, Zero Math Jargon) */}
          <div className={`p-6 rounded-3xl border relative overflow-hidden ${
            isOwed
              ? 'bg-gradient-to-br from-[#11182B] to-[#0E202A] border-emerald-500/30'
              : 'bg-gradient-to-br from-[#11182B] to-[#25131C] border-rose-500/30'
          }`}>
            <div className="flex justify-between items-start">
              <span className="text-[11px] font-bold uppercase tracking-wider text-slate-400">
                Your Personal Balance
              </span>
              <div className={`w-8 h-8 rounded-xl flex items-center justify-center ${
                isOwed ? 'bg-emerald-500/20 text-emerald-400' : 'bg-rose-500/20 text-rose-400'
              }`}>
                {isOwed ? <ArrowDownLeft className="w-4 h-4" /> : <ArrowUpRight className="w-4 h-4" />}
              </div>
            </div>

            <div className="mt-3">
              <span className="text-[10px] font-bold uppercase tracking-wider text-slate-400">
                {isOwed ? 'You Get Back' : 'You Need To Pay'}
              </span>
              <p className={`text-3xl font-extrabold tracking-tight mt-0.5 ${
                isOwed ? 'text-emerald-400' : 'text-rose-400'
              }`}>
                {isOwed ? `+₹${absBalance}` : `-₹${absBalance}`}
              </p>
            </div>

            <p className="mt-2 text-[11px] text-slate-400 leading-snug">
              {isOwed
                ? 'You paid more than your fair share. Flatmates owe you this money.'
```

```
156                : 'You paid less than your share of shared bills. Pay flatmates to settle.'}
157            </p>
158          </div>
159
160          {/* Card 2: Total House Expenses */}
161          <div className="bg-[#11182B] p-6 rounded-3xl border border-white/5 relative">
162            <div className="flex justify-between items-start">
163              <span className="text-[11px] font-bold uppercase tracking-wider text-slate-400">
164                Total Household Bills
165              </span>
166              <div className="w-8 h-8 rounded-xl bg-indigo-500/20 text-indigo-400 flex items-center justify-
center">
167                <Receipt className="w-4 h-4" />
168              </div>
169            </div>
170            <div className="mt-3">
171              <span className="text-[10px] font-bold uppercase tracking-wider text-slate-400">All Roommates
Combined</span>
172              <p className="text-3xl font-extrabold text-white tracking-tight mt-0.5">
173                ₹{kpis.totalExpenses.toLocaleString('en-IN')}
174              </p>
175            </div>
176            <p className="mt-2 text-[11px] text-slate-400">
177              Across {recentExpenses.length} shared bills this month
178            </p>
179          </div>
180
181          {/* Card 3: Your Fair Share */}
182          <div className="bg-[#11182B] p-6 rounded-3xl border border-white/5 relative">
183            <div className="flex justify-between items-start">
184              <span className="text-[11px] font-bold uppercase tracking-wider text-slate-400">
185                Your Fair Share
186              </span>
187              <div className="w-8 h-8 rounded-xl bg-purple-500/20 text-purple-400 flex items-center justify-
center">
188                <CreditCard className="w-4 h-4" />
189              </div>
190            </div>
191            <div className="mt-3">
192              <span className="text-[10px] font-bold uppercase tracking-wider text-slate-400">Equal 1/4th
Allocation</span>
193              <p className="text-3xl font-extrabold text-indigo-300 tracking-tight mt-0.5">
194                ₹{Math.round(kpis.totalExpenses / (kpis.totalMembers || 4)).toLocaleString('en-IN')}
195              </p>
196            </div>
197            <p className="mt-2 text-[11px] text-slate-400">
198              You paid ₹{myBalance.totalPaid.toLocaleString('en-IN')} so far
199            </p>
200          </div>
201
202          {/* Card 4: House Duties */}
203          <div className="bg-[#11182B] p-6 rounded-3xl border border-white/5 relative">
204            <div className="flex justify-between items-start">
205              <span className="text-[11px] font-bold uppercase tracking-wider text-slate-400">
206                House Chores
207              </span>
208              <div className="w-8 h-8 rounded-xl bg-emerald-500/20 text-emerald-400 flex items-center justify-
center">
209                <CheckCircle2 className="w-4 h-4" />
210              </div>
211            </div>
212            <div className="mt-3">
213              <span className="text-[10px] font-bold uppercase tracking-wider text-slate-400">Tasks
Completed</span>
214              <p className="text-3xl font-extrabold text-white tracking-tight mt-0.5">
215                {kpis.completedChores} <span className="text-lg text-slate-500 font-normal">/
{kpis.completedChores + kpis.pendingChores}</span>
```

```
            </p>
          </div>
          <p className="mt-2 text-[11px] text-emerald-400 font-medium">
            {kpis.pendingChores} chores pending this week
          </p>
        </div>

      </div>

      {/* Main Content Tabs (Eliminates Clutter!) */}
      <div className="space-y-6">

        {/* Tab Toggle Navigation */}
        <div className="flex space-x-2 border-b border-white/10 pb-3">
          <button
            onClick={() => setActiveTab('expenses')}
            className={`flex items-center space-x-2 px-5 py-2.5 rounded-2xl text-xs sm:text-sm font-bold
transition ${
              activeTab === 'expenses'
                ? 'bg-gradient-to-r from-indigo-500 to-purple-600 text-white shadow-md'
                : 'text-slate-400 hover:text-white hover:bg-white/5'
            }`}
          >
            <Receipt className="w-4 h-4" />
            <span>💎 Expenses & Who Owes Who</span>
          </button>

          <button
            onClick={() => setActiveTab('chores')}
            className={`flex items-center space-x-2 px-5 py-2.5 rounded-2xl text-xs sm:text-sm font-bold
transition ${
              activeTab === 'chores'
                ? 'bg-gradient-to-r from-indigo-500 to-purple-600 text-white shadow-md'
                : 'text-slate-400 hover:text-white hover:bg-white/5'
            }`}
          >
            <CheckCircle2 className="w-4 h-4" />
            <span>🧹 House Duties & Chores</span>
          </button>
        </div>

        {/* TAB 1: EXPENSES & WHO OWES WHO */}
        {activeTab === 'expenses' && (
          <div className="grid grid-cols-1 lg:grid-cols-12 gap-8">

            {/* Left: Who Owes Who (The Plain English Ledger) */}
            <div className="lg:col-span-6 bg-[#11182B] p-6 rounded-3xl border border-white/5 space-y-4">
              <div>
                <h2 className="text-lg font-bold text-white flex items-center space-x-2">
                  <span>Roommate Balance Ledger</span>
                </h2>
                <p className="text-xs text-slate-400 mt-1">
                  How much each flatmate has paid and whether they owe or get money back.
                </p>
              </div>

              <div className="space-y-3 pt-2">
                {balanceData.map((member) => {
                  const owes = member.netBalance < 0;
                  const isMe = member.id === currentUser.id;
                  const absVal = Math.abs(member.netBalance).toLocaleString('en-IN');
                  return (
                    <div
                      key={member.id}
                      className={`flex items-center justify-between p-4 rounded-2xl border transition ${
                        isMe
                          ? 'bg-white/5 border-indigo-500/30 shadow-sm'
```

```
                          : 'bg-[#172038]/50 border-white/5'
                      }`}
                    >
                      <div className="flex items-center space-x-3">
                        <div className="w-9 h-9 rounded-xl bg-gradient-to-tr from-indigo-500 to-purple-600
flex items-center justify-center text-white text-xs font-bold">
                          {member.name.slice(0, 2).toUpperCase()}
                        </div>
                        <div>
                          <p className="text-sm font-bold text-white flex items-center">
                            {member.name}
                            {isMe && <span className="ml-2 text-[10px] text-indigo-400 font-bold bg-indigo-
500/10 px-1.5 py-0.5 rounded">You</span>}
                          </p>
                          <p className="text-xs text-slate-400">
                            Paid ₹{member.totalPaid.toLocaleString('en-IN')} in total
                          </p>
                        </div>
                      </div>

                      <div className="text-right">
                        <div className={`inline-flex items-center px-2.5 py-1 rounded-xl text-xs font-bold ${
                          owes
                            ? 'bg-rose-500/10 text-rose-400 border border-rose-500/20'
                            : 'bg-emerald-500/10 text-emerald-400 border border-emerald-500/20'
                        }`}>
                          {owes ? `Owes ₹${absVal}` : `Gets back ₹${absVal}`}
                        </div>
                        {!isMe && owes && (
                          <button
                            onClick={() => handleSettle(member.name, member.netBalance)}
                            className="block text-[11px] text-indigo-400 hover:text-indigo-300 font-bold mt-1
text-right ml-auto"
                          >
                            Mark Settled ⚡
                          </button>
                        )}
                      </div>
                    </div>
                  );
                })}
              </div>
            </div>

            {/* Right: Recent Bills List */}
            <div className="lg:col-span-6 bg-[#11182B] p-6 rounded-3xl border border-white/5 space-y-4">
              <div className="flex justify-between items-center">
                <div>
                  <h2 className="text-lg font-bold text-white">Recent Shared Bills</h2>
                  <p className="text-xs text-slate-400 mt-1">Split equally among all 4 roommates</p>
                </div>
                <button
                  onClick={() => setExpenseModalOpen(true)}
                  className="text-xs font-bold text-emerald-400 hover:text-emerald-300"
                >
                  + Add Bill
                </button>
              </div>

              <div className="space-y-3 pt-2">
                {recentExpenses.map((exp) => (
                  <div
                    key={exp.id}
                    className="p-4 rounded-2xl bg-[#172038]/50 border border-white/5 flex items-center
justify-between"
                  >
                    <div>
```

```
                      <p className="text-sm font-bold text-white">{exp.description}</p>
                      <p className="text-xs text-slate-400 mt-0.5">
                        Paid by <strong className="text-slate-200">{exp.paid_by_name || 'Roommate'}</strong>
on {exp.expense_date}
                      </p>
                    </div>

                    <div className="text-right">
                      <p className="text-base font-extrabold text-white">
                        ₹{Number(exp.amount).toLocaleString('en-IN')}
                      </p>
                      <p className="text-[11px] text-indigo-300">
                        ₹{Math.round(Number(exp.amount) / 4).toLocaleString('en-IN')} / person
                      </p>
                    </div>
                  </div>
                ))}
              </div>
            </div>

          </div>
        )}

        {/* TAB 2: HOUSE CHORES & DUTIES */}
        {activeTab === 'chores' && (
          <div className="grid grid-cols-1 lg:grid-cols-12 gap-8">

            {/* Left: Quick Chores Checklist */}
            <div className="lg:col-span-7 bg-[#11182B] p-6 rounded-3xl border border-white/5 space-y-4">
              <div className="flex justify-between items-center">
                <div>
                  <h2 className="text-lg font-bold text-white">Household Task Checklist</h2>
                  <p className="text-xs text-slate-400 mt-1">Tap a chore to mark it complete</p>
                </div>
                <button
                  onClick={() => setChoreModalOpen(true)}
                  className="text-xs font-bold text-indigo-400 hover:text-indigo-300"
                >
                  + Add Task
                </button>
              </div>

              <div className="space-y-2.5 pt-2">
                {recentChores.map((chore) => {
                  const isDone = chore.status === 'COMPLETED';
                  return (
                    <div
                      key={chore.id}
                      onClick={() => toggleTask(chore.id)}
                      className={`p-4 rounded-2xl border cursor-pointer transition flex items-center justify-
between ${
                        isDone
                          ? 'bg-emerald-500/5 border-emerald-500/20 opacity-80'
                          : 'bg-[#172038]/50 border-white/5 hover:border-indigo-500/30'
                      }`}
                    >
                      <div className="flex items-center space-x-3.5">
                        <div className={`w-6 h-6 rounded-lg flex items-center justify-center border transition
${
                          isDone
                            ? 'bg-emerald-500 border-emerald-400 text-white'
                            : 'border-white/20 bg-white/5 text-transparent hover:border-indigo-400'
                        }`}>
                          <Check className="w-3.5 h-3.5" />
                        </div>
                        <div>
                          <p className={`text-sm font-semibold ${isDone ? 'line-through text-slate-400' :
```

```
'text-white'}`}>
                              {chore.title}
                            </p>
                            <p className="text-xs text-slate-400 mt-0.5">
                              Assigned to: <strong className="text-slate-300">{chore.assigned_to_name ||
'Roommate'}</strong> · Due {chore.due_date}
                            </p>
                          </div>
                        </div>

                        <div className="text-right">
                          <span className={`px-2 py-0.5 rounded-full text-[10px] font-bold ${
                            isDone
                              ? 'bg-emerald-500/10 text-emerald-400'
                              : 'bg-amber-500/10 text-amber-400'
                          }`}>
                            {isDone ? 'Done ✓' : 'Pending'}
                          </span>
                        </div>
                      </div>
                    );
                  })}
                </div>
              </div>

              {/* Right: Fair Effort Distribution Chart */}
              <div className="lg:col-span-5 bg-[#11182B] p-6 rounded-3xl border border-white/5 space-y-4">
                <div>
                  <h2 className="text-lg font-bold text-white">Chore Contribution Rank</h2>
                  <p className="text-xs text-slate-400 mt-1">Total chores completed by each flatmate</p>
                </div>

                <div className="h-64 w-full pt-4">
                  <ResponsiveContainer width="100%" height="100%">
                    <BarChart data={pointsData} margin={{ top: 10, right: 10, left: -20, bottom: 0 }}>
                      <XAxis dataKey="name" stroke="#64748B" fontSize={11} tickLine={false} />
                      <YAxis stroke="#64748B" fontSize={11} tickLine={false} />
                      <Tooltip
                        contentStyle={{ backgroundColor: '#0D1322', borderColor: '#233054', borderRadius:
'14px', color: '#fff' }}
                        formatter={(val) => [`${val} points`, 'Contribution']}
                      />
                      <Bar dataKey="points" radius={[8, 8, 0, 0]}>
                        {pointsData.map((_, index) => (
                          <Cell key={`cell-${index}`} fill={['#6366F1', '#10B981', '#F59E0B', '#8B5CF6'][index %
4]} />
                        ))}
                      </Bar>
                    </BarChart>
                  </ResponsiveContainer>
                </div>

                <p className="text-[11px] text-slate-400 text-center">
                  Lakshya and Rahul lead the household contribution leaderboard this week.
                </p>
              </div>

            </div>
          )}

        </div>

        {/* Modals */}
        {choreModalOpen && <AddChoreModal isOpen={choreModalOpen} onClose={() => setChoreModalOpen(false)} />}
        {expenseModalOpen && <AddExpenseModal isOpen={expenseModalOpen} onClose={() =>
setExpenseModalOpen(false)} />}

```

```
    </div>
  );
}
```

9. Expense Management Component *[frontend/src/pages/ExpensesPage.jsx]*

Shared bill entry modal, split previews, transaction history

```
import React, { useState } from 'react';
import { useApp } from '../context/AppContext';
import {
  Receipt,
  Plus,
  Trash2,
  Calendar,
  Filter,
  CheckCircle2,
  Users,
  CreditCard,
  Zap,
  Wifi,
  ShoppingCart,
  Flame,
  Wrench
} from 'lucide-react';
import AddExpenseModal from '../components/AddExpenseModal';

export default function ExpensesPage() {
  const { expenses, currentUser, household, deleteExpense, settleUp } = useApp();
  const [modalOpen, setModalOpen] = useState(false);
  const [filterCategory, setFilterCategory] = useState('ALL');

  const categories = [
    { id: 'ALL', name: 'All Bills', icon: Receipt },
    { id: 'Utilities', name: '⚡ Electricity & Power', icon: Zap },
    { id: 'Internet', name: '📶 Wi-Fi & Fiber', icon: Wifi },
    { id: 'Food', name: '🛒 Groceries & Food', icon: ShoppingCart },
    { id: 'Supplies', name: '🛠️ Repairs & RO', icon: Wrench },
  ];

  const filteredExpenses = filterCategory === 'ALL'
    ? expenses
    : expenses.filter(e => e.category === filterCategory);

  const totalAmount = expenses.reduce((sum, e) => sum + Number(e.amount), 0);

  return (
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8 space-y-8">

      {/* Top Banner */}
      <div class="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-4 bg-[#11182B] p-6 sm:p-8 rounded-3xl border border-white/10 shadow-2xl">
        <div>
          <h1 class="text-2xl sm:text-3xl font-extrabold text-white tracking-tight font-display">
            Shared Household Expenses
          </h1>
          <p class="mt-1 text-sm text-slate-400">
            Every bill is split equally across roommates. Zero manual math or delayed payments.
          </p>
        </div>

        <button
          onClick={() => setModalOpen(true)}
          class="flex items-center space-x-2 px-6 py-3.5 rounded-2xl bg-gradient-to-r from-emerald-500 to-teal-600 hover:from-emerald-600 hover:to-teal-700 text-white font-bold text-sm shadow-lg shadow-emerald-500/20 transition-all transform hover:-translate-y-0.5"
```

```
          >
            <Plus className="w-4 h-4" />
            <span>Add New Bill</span>
          </button>
        </div>

        {/* Summary KPI Strip */}
        <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
          <div className="bg-[#11182B] p-5 rounded-2xl border border-white/5">
            <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">Total Bills This Month</p>
            <p className="text-2xl font-extrabold text-white mt-1">₹{totalAmount.toLocaleString('en-IN')}</p>
            <p className="text-xs text-slate-400 mt-1">{expenses.length} shared records</p>
          </div>

          <div className="bg-[#11182B] p-5 rounded-2xl border border-white/5">
            <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">Equal Share Per Person</p>
            <p className="text-2xl font-extrabold text-indigo-400 mt-1">₹{Math.round(totalAmount /
4).toLocaleString('en-IN')}</p>
            <p className="text-xs text-slate-400 mt-1">Divided among 4 flatmates</p>
          </div>

          <div className="bg-[#11182B] p-5 rounded-2xl border border-white/5">
            <p className="text-xs font-bold text-slate-400 uppercase tracking-wider">Active Household</p>
            <p className="text-lg font-bold text-slate-200 mt-1 truncate">{household.name}</p>
            <p className="text-xs text-emerald-400 font-bold mt-1">Code: {household.invite_code}</p>
          </div>
        </div>

        {/* Category Pills */}
        <div className="flex flex-wrap gap-2 pt-2">
          {categories.map((c) => {
            const isActive = filterCategory === c.id;
            return (
              <button
                key={c.id}
                onClick={() => setFilterCategory(c.id)}
                className={`px-4 py-2 rounded-xl text-xs font-bold transition flex items-center space-x-1.5 ${
                  isActive
                    ? 'bg-gradient-to-r from-indigo-500 to-purple-600 text-white shadow-md'
                    : 'bg-[#11182B] text-slate-400 hover:text-white border border-white/5 hover:bg-white/5'
                }`}
              >
                <span>{c.name}</span>
              </button>
            );
          })}
        </div>

        {/* Bills Cards List */}
        <div className="space-y-3.5">
          {filteredExpenses.map((expense) => {
            const isPayer = expense.paid_by === currentUser.id;
            const share = Math.round(Number(expense.amount) / 4);

            return (
              <div
                key={expense.id}
                className="bg-[#11182B] p-5 sm:p-6 rounded-3xl border border-white/5 hover:border-white/10
transition-all flex flex-col sm:flex-row items-start sm:items-center justify-between gap-4"
              >
                <div className="flex items-center space-x-4">
                  <div className="w-12 h-12 rounded-2xl bg-indigo-500/10 border border-indigo-500/20 flex items-
center justify-center text-indigo-400">
                    <Receipt className="w-6 h-6" />
                  </div>
                  <div>
                    <h3 className="text-base font-bold text-white">{expense.description}</h3>
```

```
                  <div className="flex flex-wrap items-center gap-2 mt-1 text-xs text-slate-400">
                    <span className="font-semibold text-slate-300">
                      Paid by {expense.paid_by_name || 'Roommate'}
                    </span>
                    <span>·</span>
                    <span>{expense.expense_date}</span>
                    <span>·</span>
                    <span className="px-2 py-0.5 rounded-md bg-white/5 text-slate-300 border border-white/5">
                      {expense.category}
                    </span>
                  </div>
                </div>
              </div>

              <div className="flex items-center justify-between w-full sm:w-auto sm:space-x-6 pt-3 sm:pt-0
border-t sm:border-t-0 border-white/5">
                <div className="text-left sm:text-right">
                  <p className="text-xl font-extrabold text-white">
                    ₹{Number(expense.amount).toLocaleString('en-IN')}
                  </p>
                  <p className="text-xs text-indigo-300 font-medium">
                    {isPayer ? `You paid · You get back ₹${Number(expense.amount) - share}` : `Your share:
₹${share}`}
                  </p>
                </div>

                <button
                  onClick={() => deleteExpense(expense.id)}
                  title="Delete bill"
                  className="p-2.5 rounded-xl bg-white/5 hover:bg-rose-500/10 text-slate-500 hover:text-rose-
400 transition"
                >
                  <Trash2 className="w-4 h-4" />
                </button>
              </div>
            </div>
          );
        })}
      </div>

      {modalOpen && <AddExpenseModal isOpen={modalOpen} onClose={() => setModalOpen(false)} />}
    </div>
  );
}
```

10. Chore Task Allocation Component *[frontend/src/pages/TasksPage.jsx]*

Task list, effort-weighted assignment, status badges

```
import React, { useState } from 'react';
import { useApp } from '../context/AppContext';
import {
  CheckCircle2,
  Plus,
  Trash2,
  Calendar,
  Check,
  Clock,
  Sparkles,
  Filter
} from 'lucide-react';
import AddChoreModal from '../components/AddChoreModal';

export default function TasksPage() {
  const { tasks, currentUser, household, toggleTask, deleteTask } = useApp();
  const [modalOpen, setModalOpen] = useState(false);
```

```
 18    const [filter, setFilter] = useState('ALL'); // 'ALL', 'PENDING', 'COMPLETED', 'MY'
 19
 20    const filteredTasks = tasks.filter((t) => {
 21      if (filter === 'PENDING') return t.status === 'PENDING';
 22      if (filter === 'COMPLETED') return t.status === 'COMPLETED';
 23      if (filter === 'MY') return t.assigned_to === currentUser.id;
 24      return true;
 25    });
 26
 27    const completedCount = tasks.filter(t => t.status === 'COMPLETED').length;
 28    const pendingCount = tasks.filter(t => t.status === 'PENDING').length;
 29    const progressPercent = Math.round((completedCount / (tasks.length || 1)) * 100);
 30
 31    return (
 32      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8 space-y-8">
 33
 34        {/* Top Banner */}
 35        <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-4 bg-[#11182B]
p-6 sm:p-8 rounded-3xl border border-white/10 shadow-2xl">
 36          <div>
 37            <h1 className="text-2xl sm:text-3xl font-extrabold text-white tracking-tight font-display">
 38              House Duties & Chores
 39            </h1>
 40            <p className="mt-1 text-sm text-slate-400">
 41              Keep the living space clean without arguments. Tap any chore to mark it completed.
 42            </p>
 43          </div>
 44
 45          <button
 46            onClick={() => setModalOpen(true)}
 47            className="flex items-center space-x-2 px-6 py-3.5 rounded-2xl bg-gradient-to-r from-indigo-500 to-
purple-600 hover:from-indigo-600 hover:to-purple-700 text-white font-bold text-sm shadow-lg shadow-indigo-500/20
transition-all transform hover:-translate-y-0.5"
 48          >
 49            <Plus className="w-4 h-4" />
 50            <span>Add New Chore</span>
 51          </button>
 52        </div>
 53
 54        {/* Progress & Stat Cards */}
 55        <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
 56          <div className="bg-[#11182B] p-5 rounded-2xl border border-white/5 sm:col-span-2 flex flex-col
justify-between">
 57            <div className="flex justify-between items-center mb-3">
 58              <span className="text-xs font-bold text-slate-400 uppercase tracking-wider">Weekly House Duty
Progress</span>
 59              <span className="text-sm font-extrabold text-emerald-400">{progressPercent}% Done</span>
 60            </div>
 61            {/* Progress Bar */}
 62            <div className="w-full h-3 bg-[#0D1322] rounded-full overflow-hidden p-0.5 border border-white/5">
 63              <div
 64                className="h-full bg-gradient-to-r from-indigo-500 via-purple-500 to-emerald-400 rounded-full
transition-all duration-500"
 65                style={{ width: `${progressPercent}%` }}
 66              ></div>
 67            </div>
 68            <p className="text-xs text-slate-400 mt-3">
 69              {completedCount} chores done · {pendingCount} remaining for this week
 70            </p>
 71          </div>
 72
 73          <div className="bg-[#11182B] p-5 rounded-2xl border border-white/5 flex flex-col justify-between">
 74            <span className="text-xs font-bold text-slate-400 uppercase tracking-wider">Active Flatmate</span>
 75            <p className="text-xl font-bold text-white mt-1">{currentUser.name}</p>
 76            <p className="text-xs text-indigo-300 font-semibold mt-1">
 77              {tasks.filter(t => t.assigned_to === currentUser.id && t.status === 'PENDING').length} pending
assigned to you
```

```
          </p>
        </div>
      </div>

      {/* Filter Tabs */}
      <div className="flex flex-wrap gap-2 pt-2">
        {[
          { id: 'ALL', label: `All Tasks (${tasks.length})` },
          { id: 'PENDING', label: `Pending (${pendingCount})` },
          { id: 'COMPLETED', label: `Completed (${completedCount})` },
          { id: 'MY', label: 'Assigned to Me' },
        ].map((tab) => (
          <button
            key={tab.id}
            onClick={() => setFilter(tab.id)}
            className={`px-4 py-2 rounded-xl text-xs font-bold transition ${
              filter === tab.id
                ? 'bg-gradient-to-r from-indigo-500 to-purple-600 text-white shadow-md'
                : 'bg-[#11182B] text-slate-400 hover:text-white border border-white/5 hover:bg-white/5'
            }`}
          >
            {tab.label}
          </button>
        ))}
      </div>

      {/* Tasks Checklist */}
      <div className="space-y-3">
        {filteredTasks.map((task) => {
          const isDone = task.status === 'COMPLETED';

          return (
            <div
              key={task.id}
              className={`p-5 rounded-2xl border transition-all flex items-center justify-between gap-4 ${
                isDone
                  ? 'bg-emerald-500/5 border-emerald-500/20 opacity-80'
                  : 'bg-[#11182B] border-white/5 hover:border-white/10'
              }`}
            >
              <div
                onClick={() => toggleTask(task.id)}
                className="flex items-center space-x-4 cursor-pointer flex-1"
              >
                <div
                  className={`w-7 h-7 rounded-xl flex items-center justify-center border transition-all ${
                    isDone
                      ? 'bg-emerald-500 border-emerald-400 text-white shadow-md shadow-emerald-500/20'
                      : 'border-white/20 bg-white/5 text-transparent hover:border-indigo-400'
                  }`}
                >
                  <Check className="w-4 h-4" />
                </div>

                <div>
                  <h3 className={`text-sm sm:text-base font-bold ${isDone ? 'line-through text-slate-400' :
'text-white'}`}>
                    {task.title}
                  </h3>
                  <div className="flex flex-wrap items-center gap-2 mt-1 text-xs text-slate-400">
                    <span>Assigned to <strong className="text-slate-300">{task.assigned_to_name ||
'Roommate'}</strong></span>
                    <span>·</span>
                    <span>Due {task.due_date}</span>
                    <span>·</span>
                    <span className="px-2 py-0.5 rounded-md bg-white/5 text-slate-300 border border-white/5">
                      {task.category}
```

```
                    </span>
                  </div>
                </div>
              </div>

              <div className="flex items-center space-x-3">
                <span className={`px-2.5 py-1 rounded-full text-xs font-bold ${
                  isDone
                    ? 'bg-emerald-500/10 text-emerald-400 border border-emerald-500/20'
                    : 'bg-amber-500/10 text-amber-400 border border-amber-500/20'
                }`}>
                  {isDone ? 'Completed' : 'Pending'}
                </span>

                <button
                  onClick={() => deleteTask(task.id)}
                  title="Delete chore"
                  className="p-2 rounded-xl bg-white/5 hover:bg-rose-500/10 text-slate-500 hover:text-rose-400
transition"
                >
                  <Trash2 className="w-4 h-4" />
                </button>
              </div>
            </div>
          );
        })}
      </div>

      {modalOpen && <AddChoreModal isOpen={modalOpen} onClose={() => setModalOpen(false)} />}
    </div>
  );
}
```

11. Household Admin Control Component *[frontend/src/pages/AdminPage.jsx]*

Role-based controls, member eviction, ledger reset

```
import React, { useState } from 'react';
import { useApp } from '../context/AppContext';
import { ShieldCheck, Users, Copy, Check, Key, AlertCircle, Trash2 } from 'lucide-react';

export default function AdminPage() {
  const { household, currentUser, demoUsers, switchUser } = useApp();
  const [copied, setCopied] = useState(false);

  const isAdmin = currentUser.role === 'admin';

  const copyInvite = () => {
    navigator.clipboard.writeText(household.invite_code);
    setCopied(true);
    setTimeout(() => setCopied(false), 2000);
  };

  return (
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8 space-y-6">

      {/* Header */}
      <div>
        <h1 className="text-2xl sm:text-3xl font-extrabold text-white tracking-tight font-display">
          Flat Settings & Governance
        </h1>
        <p className="text-sm text-slate-400 mt-1">
          Manage flatmates, invite codes, and group permissions.
        </p>
      </div>

```

```
30        {/* Role Notice */}
31        {!isAdmin && (
32          <div className="p-4 rounded-2xl bg-amber-500/10 border border-amber-500/20 text-amber-300 text-xs flex
items-center justify-between">
33            <div className="flex items-center space-x-2">
34              <AlertCircle className="w-4 h-4 text-amber-400" />
35              <span>
36                You are currently viewing as <strong>{currentUser.name}</strong> (Member). Admin controls belong
to Lakshya.
37              </span>
38            </div>
39            <button
40              onClick={() => switchUser('u1')}
41              className="px-3 py-1 bg-amber-500 hover:bg-amber-600 text-slate-900 rounded-lg font-bold text-xs"
42            >
43              Switch to Lakshya (Admin)
44            </button>
45          </div>
46        )}
47
48        {/* Household Profile & Invite Code Box */}
49        <div className="bg-[#11182B] p-6 sm:p-8 rounded-3xl border border-white/5 space-y-6">
50          <h3 className="font-bold text-white text-lg flex items-center space-x-2">
51            <Key className="w-5 h-5 text-indigo-400" />
52            <span>Flat Access & Credentials</span>
53          </h3>
54
55          <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
56            <div className="p-5 rounded-2xl bg-[#0D1322] border border-white/5">
57              <label className="text-[10px] font-bold text-slate-400 uppercase tracking-wider block mb-1">
58                Flat / House Name
59              </label>
60              <p className="font-bold text-white text-base">{household.name}</p>
61              <p className="text-xs text-slate-400 mt-1">Created by Lakshya Shukla</p>
62            </div>
63
64            <div className="p-5 rounded-2xl bg-[#0D1322] border border-white/5 flex items-center justify-
between">
65              <div>
66                <label className="text-[10px] font-bold text-slate-400 uppercase tracking-wider block mb-1">
67                  Roommate Invite Code
68                </label>
69                <p className="font-mono font-extrabold text-2xl text-emerald-400">{household.invite_code}</p>
70                <p className="text-xs text-slate-400 mt-1">Share this code with your flatmates</p>
71              </div>
72
73              <button
74                onClick={copyInvite}
75                className="flex items-center space-x-1.5 px-4 py-2.5 rounded-xl bg-white/5 hover:bg-white/10
text-slate-200 text-xs font-bold border border-white/10 transition"
76              >
77                {copied ? <Check className="w-4 h-4 text-emerald-400" /> : <Copy className="w-4 h-4" />}
78                <span>{copied ? 'Copied!' : 'Copy Code'}</span>
79              </button>
80            </div>
81          </div>
82        </div>
83
84        {/* Roommates Roster Table */}
85        <div className="bg-[#11182B] p-6 sm:p-8 rounded-3xl border border-white/5 space-y-6">
86          <div className="flex justify-between items-center">
87            <div>
88              <h3 className="font-bold text-white text-lg flex items-center space-x-2">
89                <Users className="w-5 h-5 text-indigo-400" />
90                <span>Flatmate Roster (4 Members)</span>
91              </h3>
92              <p className="text-xs text-slate-400 mt-1">Active flatmates sharing expenses and duties</p>
```

```
          </div>
        </div>

        <div className="overflow-x-auto">
          <table className="w-full text-left text-sm text-slate-300">
            <thead>
              <tr className="border-b border-white/5 text-[10px] font-bold text-slate-400 uppercase tracking-
wider">
                <th className="pb-3">Roommate</th>
                <th className="pb-3">Email Address</th>
                <th className="pb-3">Role</th>
                <th className="pb-3">Status</th>
                <th className="pb-3 text-right">Actions</th>
              </tr>
            </thead>
            <tbody className="divide-y divide-white/5">
              {demoUsers.map((user) => (
                <tr key={user.id} className="hover:bg-white/5 transition">
                  <td className="py-4 font-bold text-white flex items-center space-x-3">
                    <div className={`w-8 h-8 rounded-lg bg-gradient-to-tr ${user.avatarColor} flex items-
center justify-center text-white text-xs font-bold`}>
                      {user.initials}
                    </div>
                    <span>{user.name}</span>
                  </td>
                  <td className="py-4 text-slate-400 font-mono text-xs">{user.email}</td>
                  <td className="py-4">
                    <span className={`px-2.5 py-0.5 rounded-full text-[10px] font-bold ${
                      user.role === 'admin'
                        ? 'bg-indigo-500/10 text-indigo-400 border border-indigo-500/20'
                        : 'bg-white/5 text-slate-400'
                    }`}>
                      {user.role === 'admin' ? 'Flat Manager' : 'Flatmate'}
                    </span>
                  </td>
                  <td className="py-4">
                    <span className="inline-flex items-center text-xs text-emerald-400 font-bold">
                      <span className="w-1.5 h-1.5 rounded-full bg-emerald-400 mr-1.5"></span>
                      Active
                    </span>
                  </td>
                  <td className="py-4 text-right">
                    {user.role !== 'admin' && (
                      <button
                        disabled={!isAdmin}
                        className="text-xs text-rose-400 hover:text-rose-300 font-bold disabled:opacity-30
disabled:cursor-not-allowed"
                      >
                        Remove
                      </button>
                    )}
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>

    </div>
  );
}
```

12. Vite Build Tooling Configuration *[frontend/vite.config.js]*

Vite plugins, asset bundling, API reverse proxy

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  base: './',
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:5000',
        changeOrigin: true
      }
    }
  }
});
```

13. Vercel Edge SPA Routing Rules *[frontend/vercel.json]*

Production SPA URL rewrite configuration

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```